

Project SixEyes

AI execution-cost forensics — read-only waste detection for agentic LLM workloads

DOCUMENT ID	PROJECT STATUS	CLASSIFICATION
PS-SIXEYES-001	Pre-development / Design	Internal — Confidential
OWNER	DATE	VERSION
Founder / Product Owner	16 September 2026	1.1

1. Background

Organizations running LLM-based applications — chat assistants, retrieval-augmented generation (RAG) systems, and increasingly autonomous coding/research agents — are seeing inference cost grow faster than infrastructure teams can track it. Industry data through 2026 puts agentic workflows at 5–30x the token cost of a simple chat query, driven by tool calls, multi-step reasoning, retries, and repeatedly re-sent context. Independent audits attribute the majority of this overhead to structural inefficiency rather than genuine task complexity — most notably, broken prompt-cache reuse, which alone has been shown to move total LLM spend by 50–80% in production systems with no change to output quality.

Provider-native tooling is starting to close part of this gap for a single request pair on a single provider — Anthropic, for example, ships a beta API that fingerprints consecutive requests and reports the exact divergence point at no cost. What remains unaddressed by any provider-native feature or observability platform (Langfuse, Helicone, Datadog LLM Observability) is aggregation across providers, prioritization by dollar impact across a fleet of traffic over time, and the non-cache waste categories (redundant tool calls, re-sent context, retry storms) that a single-request cache check does not touch. That narrower, honest gap is what this project tests.

2. Problem Statement

Teams running LLM applications at meaningful scale are overpaying for inference — often by 30–70% — because of structural, unintentional waste in how requests are constructed and executed (unstable prompt prefixes breaking provider caching, redundant tool calls, re-sent conversation history, retry storms). This waste is invisible in standard dashboards, which report aggregate cost and latency but not **root cause** or **a specific, verifiable fix**. Teams either overpay silently, or attempt manual optimization that risks degrading output quality with no way to measure that risk before shipping.

3. Objective

Build a system that ingests a team’s LLM execution traces and automatically:

- **Detects** specific, named categories of recoverable waste (starting with broken prompt-cache prefixes, redundant tool calls, and re-sent context) — deterministically, with no LLM involved in the detection logic itself.
- **Localizes** the exact cause — the field, position, or code path responsible — not just the aggregate metric.
- **Quantifies** the recoverable cost in dollars, using a fully auditable calculation.
- **Prescribes** a concrete fix, and later verifies the savings after the customer applies it.

The governing constraint: the system diagnoses and recommends — it never modifies a production request. That boundary is provable and is the thing this is a diagnostic tool, not a quality/cost trade-off tool, rests on. What it does **not**

claim: that applying a recommended fix is guaranteed to leave model output unchanged. Reordering a tool array or moving a timestamp to repair a broken cache prefix is itself a change to what the model sees. Every reported number is labelled as one of four tiers — measured (a real before/after), derived (exact arithmetic on observed data), estimated (modelled), or speculative (advisory text only) — and only "measured" is ever presented as a result.

4. Scope

IN SCOPE (first build)	OUT OF SCOPE (for now)
• Ingest execution traces from common formats (OpenTelemetry GenAI conventions, direct provider SDK logs, plain exported logs)	• Acting on findings automatically (no changes made on the team's behalf)
• Deterministic detection of: unstable cache prefixes, redundant tool calls, duplicated/re-sent context, retry waste	• Model routing, prompt rewriting, or any capability that changes model behavior or output
• Dollar-cost attribution per finding, using versioned, dated provider pricing	• Semantic/fuzzy caching (approximate-match reuse) — quality risk, deferred
• A report/output format a human can act on directly (finding, evidence, fix, dollar impact)	• Multi-tenant hosted service, billing, and account management
• A way to confirm, after a fix is applied, that the saving actually happened	• Support for every model provider on day one — start with the two or three most common

5. Key Technical Requirements

5.1 Correctness & trust

- Detection logic must be fully deterministic and reproducible — same input trace always produces the same findings. No probabilistic/LLM-based judgment in the detection path.
- Every reported finding must carry machine-checkable evidence, not just a conclusion (e.g. "divergence at message index 2, token offset 1,847" — not "cache looks broken").
- Dollar figures must use exact arithmetic (no floating-point currency) and cite which priced-per-token table version produced them, since provider prices change monthly.

5.2 Data handling

- The system must never need to store or transmit a customer's actual prompt or completion text to produce a finding — detection should work on structural fingerprints (hashes, token counts, position offsets) derived from the trace, not the trace content itself.
- This needs to be provable, not just claimed — i.e., an engineer should be able to audit exactly what leaves the customer's environment.

5.3 Extensibility

- New waste-detection categories will be added over time (redundant tool calls, retry storms, oversized context, etc.). Adding one must not require touching or re-running the existing ones.
- Given that traces can be large, re-analysis after adding a new detector should reuse prior computation rather than reprocessing everything from scratch.

5.4 Explainability of fixes

- A human-written or human-readable explanation of *how to fix* a finding is useful, but must be clearly separated from the deterministic finding itself — the dollar figure and evidence must never depend on this explanation being correct.

6. Deliverables & Definition of Done

#	Deliverable	Definition of done
1	Trace ingestion	Can read a real trace export and normalize it into one internal format, regardless of source.
2	Content-free fingerprinting	Two requests differing by one dynamic field (e.g. a timestamp) are shown to diverge at the correct position — with an auditable proof that no prompt content was retained to do it.
3	Waste detectors (v1 set)	Each detector finds a deliberately-inserted defect in a test case, 100% of the time, and produces zero false positives on a clean trace.
4	Cost attribution	Reported recoverable-dollar total never exceeds the trace's actual total spend; overlapping causes are not double-counted.
5	Report output	A person with no context can open the output and understand: what's wrong, proof it's wrong, what to change, and how many dollars it's worth.
6	Validation on real data	Run against at least 5 real workloads (not synthetic). At least 3 must show meaningful recoverable spend before this is considered validated.

7. Constraints & Assumptions

- **No quality trade-offs in v1.** Anything that could change model output quality (model swapping, prompt rewriting, semantic caching) is explicitly deferred — it requires an evaluation framework that does not exist yet.
- **No production traffic interception.** The system reads exported/logged traces; it does not sit between the application and the model provider in this phase.
- **Small team, limited timeline.** Build order matters — the core analysis engine must be solid before adding detectors, and detectors must be validated on real data before adding a user-facing product layer around them.
- **Pricing data will go stale.** Provider token pricing must be treated as versioned, dated input, not a hardcoded constant.

8. Open Questions for Engineering Review

- What's the right internal architecture for a system with a fixed core pipeline (ingest → fingerprint) but a growing, independently-testable set of detection modules that must run in parallel and support incremental re-analysis?
- What's the most defensible way to prove, technically, that no raw prompt content is ever retained or transmitted by the fingerprinting step?
- How should confidence be represented when a dollar figure is a measured fact (proven by before/after data) versus a modeled estimate — and how do we stop the two from being presented the same way?
- What trace formats and providers should the first version support, given limited engineering time?

End of document. Prepared as an internal technical brief for engineering review.